SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1711**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: 2 **Duration : 45 Minutes**

Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

*I **SHABARISH N 1924242007** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Model Answers

Answer 1: Intelligent Agents in the Smart Home System

Simple Reflex Agent

Acts only on the current percept via fixed if–then rules, with no memory. Example: "IF motion detected → turn ON light." Cannot handle partial observability or learn preferences.

Model-Based Reflex Agent

Keeps an internal state to track things not directly observable. Example: remembers a room is "occupied" during brief gaps in motion detection, avoiding flickering lights.

Goal-Based Agent

Chooses actions by searching/planning to reach an explicit goal. Example: plans heater activation time to reach 22°C by 10 p.m.

Utility-Based Agent

Assigns a utility score to outcomes and picks the action maximising it, allowing trade-offs. Example: dims lights instead of switching off to balance comfort and energy saving; weighs comfort, energy, and security together.

Best Approach

A hybrid, learning agent works best: a model-based core for situational awareness, a utility-based layer to balance comfort/energy/security trade-offs, simple-reflex rules kept only for instant safety alarms, and a learning component so the utility weights adapt to the user's preferences over time.

Answer 2: Uninformed Search for Robot Maze Navigation

Uniform Cost Search (UCS)

Expands the lowest cumulative-cost node first (priority queue). Guarantees the cheapest path to the exit, even with varying step costs, but is complete/optimal at the price of high memory use.

Iterative Deepening Search (IDS)

Repeats depth-limited DFS with an increasing depth limit. Combines DFS's low memory use with BFS-like completeness, and is optimal when all step costs are equal.

Algorithm	Time	Space	Key Trade-off
UCS	O(b^(1+C*/ε))	O(b^(1+C*/ε))	Optimal with variable costs; high memory use
IDS	O(b^d)	O(bd)	Memory-efficient; re-expands nodes; needs uniform costs for optimality

Relation to Agent Types

The robot is a goal-based agent (goal = reach the exit) that needs a model-based memory of visited cells to run any search at all, since it must track what has already been explored to avoid infinite loops.

Recommended Combination

A model-based, goal-based agent using IDS is best by default — low memory footprint suits an unknown maze and it stays optimal when step costs are uniform. If step costs vary (e.g., rough terrain), switch to UCS, since only UCS guarantees the cheapest path under unequal costs.